

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

SORTING I

WE CAN SEARCH FAST. BUT SOMEONE HAS TO SORT FIRST.

SETTLED

Binary search costs $\Theta(\log n)$. We can write a recurrence for a recursive algorithm and unroll it into a closed form.

OPEN

Binary search only works on a sorted array. Nobody handed us one. Producing it is its own problem, with its own cost.

Sorting is the price of that assumption.

WHY SORTING IS IMPORTANT

ANALYSIS PRACTICE

Three loop invariants in one algorithm, and the first recursive running time you will derive yourself.

YOU WILL PICK ONE

Almost nobody implements a sort at work. Everybody chooses one. That choice needs a reason.

DESIGN STRATEGIES

Incremental construction and divide and conquer both show up here first. The strategies outlive the algorithms.

SORTED DATA IS CHEAP

Median in $\Theta(1)$. Search in $\Theta(\log n)$. Sorting is a preprocessing step that pays for itself.

SELECTION SORT

THE PROCEDURE YOU WOULD INVENT ANYWAY

Spread five cards on a table. Repeatedly take the smallest card that is left and place it at the end of a new row.

UNSORTED	SORTED OUTPUT
7 2 1 -3 9	
7 2 1 9	-3
7 2 9	-3 1
7 9	-3 1 2
9	-3 1 2 7
	-3 1 2 7 9

The output row is sorted at every step, not only at the end.

THE OBVIOUS IMPLEMENTATION WASTES A LIST

TWO LISTS

Pop the minimum from the input, append it to a fresh output list. Correct, and easy to see why. It also allocates $\Theta(n)$ extra memory we do not need.

ONE LIST, ONE BARRIER

Keep a single list split into two parts. The head is sorted and final. The tail is untouched. The barrier is the index that separates them.

sorted, in final position

not yet examined

THE PROCEDURE YOU WOULD INVENT ANYWAY

Spread five cards on a table. Go index by index. Take the smallest card and swap with the current index.

	SORTED	UNSORTED
Iteration 0:		7 2 1 -3 9
Iteration 1:	-3	2 1 7 9
Iteration 2:	-3 1	2 7 9
Iteration 3:	-3 1 2	7 9
Iteration 4:	-3 1 2 7	9
Iteration 5:	-3 1 2 7 9	

IN-PLACE SELECTION SORT

```
def selection_sort(arr):  
    """In-place selection sort."""  
    n = len(arr)  
    if n <= 1:  
        return  
    for barrier_ix in range(n-1):  
        min_ix = find_minimum(arr, start=barrier_ix)  
        arr[barrier_ix], arr[min_ix] = (  
            arr[min_ix], arr[barrier_ix])
```

```
def find_minimum(arr, start):  
    """Assumes arr is non-empty."""  
    n = len(arr)  
    min_value = arr[start]  
    min_ix = start  
    for i in range(start + 1, n):  
        if arr[i] < min_value:  
            min_value = arr[i]  
            min_ix = i  
    return min_ix
```


THREE INVARIANTS, NOT ONE

INVARIANT 1

After the α th iteration, each of the first α elements of `arr` is less than or equal to each of the last $\text{len}(\text{arr}) - \alpha$ elements.

INVARIANT 2

After the α th iteration, the first α elements of `arr` are in sorted order.

INVARIANT 3

After the α th iteration, `arr` is a permutation of its original elements.

PROVING INVARIANT 1

BASE CASE

Set $\alpha = 0$. The claim is that the first zero elements are less than or equal to the last $\text{len}(\text{arr})$ elements.

The first zero elements form an empty list. A universal statement about an empty list is vacuously true.

INDUCTIVE HYPOTHESIS

Assume it holds after iteration $\alpha - 1$.

INDUCTIVE STEP

Iteration α swaps position α with x , the minimum of the tail.

x is a minimum, so x is at most every element from α onward. The hypothesis already covers the first $\alpha - 1$. Together the first α elements are at most the last $\text{len}(\text{arr}) - \alpha$.

TERMINATION EXPLAINS THE OFF-BY-ONE

The loop runs $\text{len}(\text{arr}) - 1$ times, not $\text{len}(\text{arr})$. Plug that into all three invariants and read what falls out.

01 Each of the first $n - 1$ elements is less than or equal to the last element.

02 The first $n - 1$ elements are sorted.

03 arr holds the same elements it started with.

THE COST IS A TRIANGLE

ONE CALL

`find_minimum(arr, start)` scans from `start + 1` to `n - 1`. Its loop body runs `n - start - 1` times.

ALL CALLS

`barrier_ix` takes every value from 0 to `n - 2`, so `start` takes each of those values exactly once.

$$T(n) = (n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2 = \Theta(n^2)$$

Best case, worst case, and average case are all $\Theta(n^2)$.

Selection sort has no early exit. It scans the entire tail on every iteration regardless of what it finds there, so the input cannot help it.

INSERTION SORT

SAME BARRIER, OPPOSITE WORK

SELECTION SORT

Search the tail for the right element, then put it in the obvious place.
The search is expensive, the placement is free.

INSERTION SORT

Take the obvious element, then search the head for the right place.
The placement is expensive, the choice is free.

```
def insertion_sort(arr):  
    """In-place insertion sort."""  
    for barrier_ix in range(1, len(arr)):  
        current = arr[barrier_ix]  
        i = barrier_ix - 1  
        while i >= 0 and arr[i] > current:    # slide larger elements right  
            arr[i + 1] = arr[i]  
            i -= 1  
        arr[i + 1] = current
```

The head is still sorted after every iteration. It is just no longer in final position, because a later element can still land inside it.

INSERTION SORT CHARGES YOU FOR DISORDER

```
def insertion_sort(arr):  
    """In-place insertion sort."""  
    for barrier_ix in range(1, len(arr)):  
        current = arr[barrier_ix]  
        i = barrier_ix - 1  
        while i >= 0 and arr[i] > current:    # slide larger elements right  
            arr[i + 1] = arr[i]  
            i -= 1  
        arr[i + 1] = current
```

BEST CASE

Input already sorted. $\text{arr}[i] > \text{current}$ fails on the first test every time, so the while loop never executes a body.

$n - 1$ outer iterations, constant work each. $\Theta(n)$

WORST CASE

Input in reverse order. Every element is smaller than everything before it, so the while loop walks all the way to index 0.

$1 + 2 + \dots + (n - 1)$ shifts. $\Theta(n^2)$

TWO QUADRATIC SORTS, DIFFERENT FAILURE MODES

PROPERTY	SELECTION	INSERTION	WHY IT MATTERS
Best case	$\Theta(n^2)$	$\Theta(n)$	Nearly sorted data is common in practice.
Worst case	$\Theta(n^2)$	$\Theta(n^2)$	Neither survives a large input.
Writes to the array	$\Theta(n)$	$\Theta(n^2)$	Writes dominate when elements are large.
Adaptive	No	Yes	Can the algorithm exploit existing order?
Stable	No	Yes	Equal keys keep their original relative order.

DIVIDE AND CONQUER

THREE QUESTIONS FOR ANY RECURSIVE ALGORITHM

01 **What is the base case, and is it correct?**

Arrays of size one are trivially sorted. The function returns immediately.

02 **Are the recursive problems strictly smaller?**

Both halves of an array of size n are smaller than n . Check it at $n = 2$, where splitting is tightest.

03 **Assuming the recursive calls work, does the whole thing work?**

Given two sorted halves, does the combine step produce a sorted whole?

MERGE SORT: SPLIT, RECURSE, MERGE

```
def mergesort(arr):  
    if len(arr) > 1:  
        middle = math.floor(len(arr) / 2)  
        left = arr[:middle]  
        right = arr[middle:]  
        mergesort(left)  
        mergesort(right)  
        merge(left, right, arr)
```

DIVIDE

[6,1,9,2,4,3] → [6,1,9] [2,4,3]

CONQUER

[6,1,9] → [1,6,9] [2,4,3] → [2,3,4]

COMBINE

[1,6,9] [2,3,4] → [1,2,3,4,6,9]

MERGE IS WHERE THE SORTING HAPPENS

```
def merge(left, right, out):  
    """Merge sorted arrays, store in out."""  
    left.append(float('inf'))  
    right.append(float('inf'))  
    left_ix = 0  
    right_ix = 0  
    for ix in range(len(out)):  
        if left[left_ix] < right[right_ix]:  
            out[ix] = left[left_ix]  
            left_ix += 1  
        else:  
            out[ix] = right[right_ix]  
            right_ix += 1
```

LOOP INVARIANT

Assume left and right are sorted. After the α th iteration, $\text{out}[:\alpha]$ holds the α smallest elements of left and right, in sorted order.

THE RECURRENCE, AND THE COST HIDING IN THE SLICES

SLICING IS NOT FREE

`arr[:middle]` copies. Two slices copy n elements between them, so the split alone is $\Theta(n)$, not $\Theta(1)$.

SO THE LOCAL WORK IS LINEAR

$\Theta(n)$ to split plus $\Theta(n)$ to merge, then two recursive calls on inputs of size $n/2$.

$$\begin{aligned} T(n) &= 2 T(n/2) + n \\ &= 4 T(n/4) + 2n && \text{divide again} \\ &= 8 T(n/8) + 3n && \text{divide again} \\ &= 2^k T(n/2^k) + k \cdot n && \text{after } k \text{ steps} \end{aligned}$$

Stop when $n/2^k = 1$, so $k = \log_2 n$. Substituting gives $n \cdot T(1) + n \log_2 n$, and $T(n) = \Theta(n \log n)$.

OPTIMAL?

THEOREM

Any comparison-based sorting algorithm has worst-case time complexity $\Omega(n \log n)$. Merge sort matches that bound, so its worst case cannot be beaten.

NOTE

The bound is about worst case.

Merge sort's best case is also $\Theta(n \log n)$, because it splits and merges the same way no matter what the input looks like.

Insertion sort's best case is $\Theta(n)$. On nearly sorted data, merge sort loses.

EXAM-STYLE

Consider a hybrid that divides once and does not recurse:

```
def mergeselectionsort(arr):  
    middle = math.floor(len(arr) / 2)  
    left = arr[:middle]  
    right = arr[middle:]  
    selection_sort(left)  
    selection_sort(right)  
    merge(left, right, arr)
```

THE QUESTION

Give the worst-case time complexity in terms of n .
Then say what it would take to reach $\Theta(n \log n)$.

EXAM-STYLE SOLUTION

Two selection sorts on $n/2$

$$2 \cdot \Theta((n/2)^2) = \Theta(n^2/2) = \Theta(n^2)$$

One merge on n elements

$$\Theta(n)$$

Total

$$\Theta(n^2) + \Theta(n) = \Theta(n^2) \quad \text{by the max rule}$$

WHAT THE SPLIT ACTUALLY BOUGHT

Halving the input before a quadratic sort cuts the constant, not the growth rate. Θ discards the constant, so the answer is unchanged.
The $\Theta(n \log n)$ in merge sort comes from recursing all the way down to size one, not from splitting once.